

Structured Program Development

C - How to Program, 9th edition

Umer Shahid

Department of Electrical Engineering,
UET Lahore

September 28, 2025

Overview

1. Introduction to Problem Solving
2. Algorithms
3. Pseudocode
4. Control Structures
5. Selection Statements
6. Iteration Statements
7. Assignment and Increment Operators
8. Problem Solving with Top-Down Design
9. Practice Problems
10. Nested Loops and Patterns

Why Do We Need Structured Programming?

- **Before Writing Code:** You need a clear plan!
- Just like building a house:
 - You don't start by mixing cement
 - You first make blueprints
 - Then follow a systematic approach
- **Programming is Problem Solving**
 - Understand the problem completely
 - Plan your solution step by step
 - Then write the code

What is an Algorithm?

Definition

An **algorithm** is a step-by-step procedure for solving a problem, consisting of:

1. The **actions** to execute
2. The **order** in which these actions should execute

Important

The ORDER matters! Wrong order = Wrong result!

Algorithm Example: Getting Ready for Work

Correct Order:

1. Get out of bed
2. Take off pajamas
3. Take a shower
4. Get dressed
5. Eat breakfast
6. Go to work

Algorithm Example: Getting Ready for Work

Correct Order:

1. Get out of bed
2. Take off pajamas
3. Take a shower
4. Get dressed
5. Eat breakfast
6. Go to work

Wrong Order:

1. Get out of bed
2. Take off pajamas
3. Get dressed
4. Take a shower
5. Eat breakfast
6. Go to work

Result: You arrive at work soaking wet!

What is Pseudocode?

Definition

Pseudocode is an informal language similar to everyday English that helps you develop algorithms before converting them to C programs.

Characteristics:

- Uses everyday English words
- Not a real programming language
- Computers cannot execute pseudocode
- Helps you "think out" your program
- Easy to convert to actual C code

Pseudocode Example

Problem: Calculate the average of three numbers

Pseudocode Solution

1. Input first number
2. Input second number
3. Input third number
4. Add all three numbers
5. Divide sum by 3
6. Display the average

Note: This is much easier to understand than C code!

What are Control Structures?

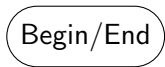
- **Sequential Execution:** Normally, statements execute one after another
- **Transfer of Control:** Sometimes we need to change this order
- **Control Structures:** Tools that help us control the flow of execution

The Three Types of Control Structures

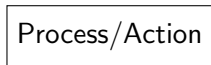
1. **Sequence:** Execute statements one by one
2. **Selection:** Choose between different paths
3. **Iteration:** Repeat statements multiple times

Flowcharts - Visual Representation

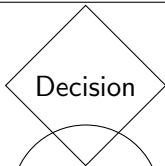
Flowcharts help us visualize algorithms using special symbols:



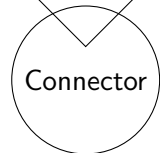
Start/End



Calculation/Input/Output

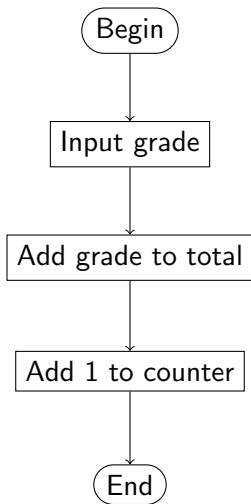


True/False Question



Connection Point

Simple Flowchart Example



This shows sequence structure - one step after another

The if Selection Statement

Purpose: Execute code only if a condition is true

Syntax

```
1 if (condition) {  
2     statement(s);  
3 } // end if
```

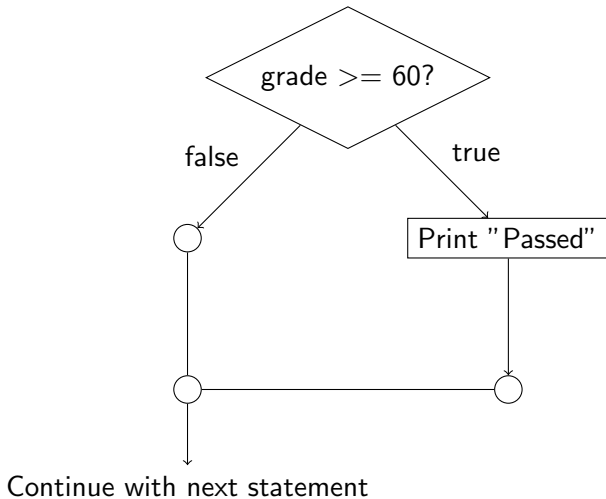
Example

```
1 if (grade >= 60) {  
2     printf("Passed\n");  
3 } // end if
```

Key Points:

- Condition must be in parentheses
- Use braces { } even for single statements
- Indentation makes code readable

if Statement Flowchart



The if...else Selection Statement

Purpose: Execute one action if condition is true, another if false

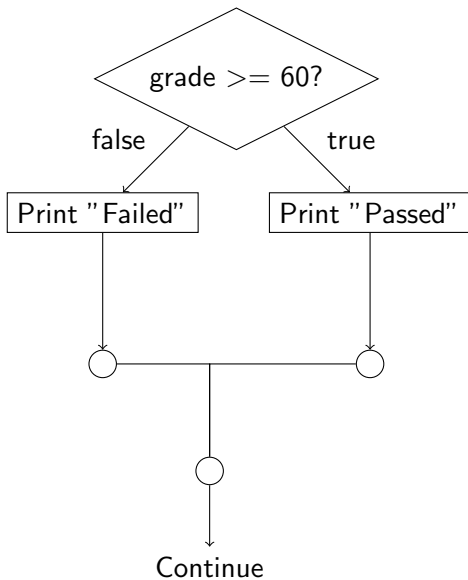
Syntax

```
1 if (condition) {  
2     statement(s) for true;  
3 } // end if  
4 else {  
5     statement(s) for false;  
6 } // end else
```

Example

```
1 if (grade >= 60) {  
2     printf("Passed\n");  
3 } // end if  
4 else {  
5     printf("Failed\n");  
6 } // end else
```

if...else Statement Flowchart



Conditional Operator (?:)

Shortcut for simple if...else statements

Syntax

```
condition ? value_if_true : value_if_false
```

Example

```
1 // Instead of:
2 if (grade >= 60) {
3     printf("Passed\n");
4 } else {
5     printf("Failed\n");
6 }
7
8 // You can write:
9 printf("%s\n", (grade >= 60) ? "Passed" : "Failed");
```

Use only for simple cases!

Nested if...else Statements

Multiple conditions require multiple if...else statements

Grade Classification Example

```
1  if (grade >= 90) {  
2      printf("A\n");  
3  } // end if  
4  else if (grade >= 80) {  
5      printf("B\n");  
6  } // end else if  
7  else if (grade >= 70) {  
8      printf("C\n");  
9  } // end else if  
10 else if (grade >= 60) {  
11     printf("D\n");  
12 } // end else if  
13 else {  
14     printf("F\n");  
15 } // end else
```

Key Point: Only the first true condition executes!

Compound Statements (Blocks)

Multiple statements can be grouped using braces { }

Example

```
1 if (grade >= 60) {  
2     printf("Passed.\n");  
3     printf("Congratulations!\n");  
4     passed_count++;  
5 } // end if  
6 else {  
7     printf("Failed.\n");  
8     printf("You must retake this course.\n");  
9     failed_count++;  
10 } // end else
```

Always Use Braces!

Even for single statements - it prevents errors and makes code clearer

The while Loop

Purpose: Repeat statements while a condition is true

Syntax

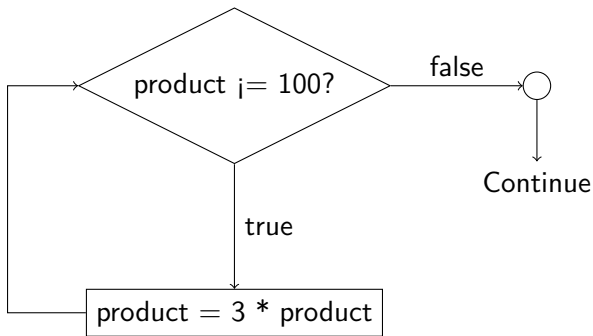
```
1 while (condition) {  
2     statement(s);  
3 } // end while
```

Example: Finding First Power of 3 $\geq$ 100

```
1 int product = 3;  
2 while (product <= 100) {  
3     product = 3 * product;  
4 }  
5 // product is now 243
```

Warning: Make sure the condition eventually becomes false, or you'll have an infinite loop!

while Loop Flowchart



Notice the loop - condition is checked every time!

Counter-Controlled Iteration

When you know exactly how many times to repeat

Example: Calculate Class Average for 10 Students

```
1 int total = 0;           // sum of grades
2 int counter = 1;         // number of grades entered
3
4 while (counter <= 10) {   // repeat 10 times
5     printf("Enter grade: ");
6     int grade;
7     scanf("%d", &grade);
8     total = total + grade;
9     counter = counter + 1;
10 } // end while
11
12 int average = total / 10;
13 printf("Class average is %d\n", average);
```

Key Variables: counter (counts iterations), total (accumulates sum)

Important: Initialize Your Variables

Common Mistake

Forgetting to initialize counter and total variables!

Correct Initialization

```
1 int total = 0;           // MUST start at 0 for addition
2 int counter = 1;         // Start at 1 (first student)
3 // int counter = 0;      // OR start at 0, then use counter < 10
```

Why?

- Uninitialized variables contain "garbage" values
- $\text{total} = \text{garbage} + \text{grade} = \text{wrong answer!}$
- Always initialize counters to 0 or 1
- Always initialize totals to 0

Sentinel-Controlled Iteration

When you don't know how many times to repeat

Problem: Calculate average for unknown number of grades

Solution: Use a special "sentinel" value to signal "stop"

Example: Use -1 as Sentinel

```
1 int total = 0, counter = 0;
2 printf("Enter grade, -1 to end: ");
3 int grade;
4 scanf("%d", &grade);
5
6 while (grade != -1) {           // continue until sentinel
7     total = total + grade;      // process the grade
8     counter = counter + 1;
9     printf("Enter grade, -1 to end: ");
10    scanf("%d", &grade);        // get next grade
11 } // end while
12
13 if (counter != 0) {
14     double average = (double) total / counter;
15     printf("Class average is %.2f\n", average);
16 } else {
17     printf("No grades were entered\n");
18 }
```

Why Check for Division by Zero?

```
1 if (counter != 0) {  
2     double average = (double) total / counter;  
3     printf("Class average is %.2f\n", average);  
4 } else {  
5     printf("No grades were entered\n");  
6 }
```

What if counter is 0?

- Division by zero causes program to crash!
- This is called a "fatal error"
- Always check before dividing
- Handle the error gracefully

Good Programming Practice

Always validate your data before using it!

Data Types: int vs double

Problem: Average might not be a whole number!

```
1 int total = 81;
2 int counter = 10;
3 int average = total / counter;      // Result: 8 (not 8.1!)
```

Solution: Use floating-point numbers

```
1 int total = 81;
2 int counter = 10;
3 double average = (double) total / counter;  // Result: 8.1
4 printf("Average: %.2f\n", average);         // Shows: 8.10
```

Key Points:

- int division truncates (cuts off) decimal part
- double can store decimal numbers
- (double) is a "cast operator" - converts int to double

Formatting Output with printf

Control how numbers are displayed

Format Specifiers for double

```
1 double value = 3.14159;
2
3 printf("%.1f\n", value); // Shows: 3.1
4 printf("%.2f\n", value); // Shows: 3.14
5 printf("%.3f\n", value); // Shows: 3.142
6 printf("%f\n", value);   // Shows: 3.141590 (default 6 digits)
```

The .2 means "show 2 digits after decimal point"

- Values are rounded, not truncated
- 3.446 with %.2f shows 3.45 (rounded up)
- Very useful for displaying money: %.2f for dollars.cents

Assignment Operators

Shortcuts for common operations

Instead of writing this:

```
1 c = c + 3;  
2 d = d - 4;  
3 e = e * 5;  
4 f = f / 3;  
5 g = g % 9;
```

You can write this:

```
1 c += 3;    // same as c = c + 3  
2 d -= 4;    // same as d = d - 4  
3 e *= 5;    // same as e = e * 5  
4 f /= 3;    // same as f = f / 3  
5 g %= 9;    // same as g = g % 9
```

Increment and Decrement Operators

Special operators for adding/subtracting 1

Four Different Ways to Add 1:

```
1 counter = counter + 1;    // Long way
2 counter += 1;             // Shorter way
3 ++counter;               // Pre-increment
4 counter++;               // Post-increment
```

Four Different Ways to Subtract 1:

```
1 counter = counter - 1;    // Long way
2 counter -= 1;             // Shorter way
3 --counter;               // Pre-decrement
4 counter--;               // Post-decrement
```

Pre vs Post Increment

When used alone, they do the same thing:

```
1 ++counter;    // Increment counter
2 counter++;    // Same result when used alone
```

Different when used in expressions:

```
1 int c = 5;
2 printf("%d\n", c++);    // Prints 5, then c becomes 6
3 printf("%d\n", c);      // Prints 6
4
5 c = 5;
6 printf("%d\n", ++c);    // c becomes 6, then prints 6
7 printf("%d\n", c);      // Prints 6
```

For beginners: Use them on separate lines to avoid confusion!

```
1 printf("%d\n", c);
2 ++c;    // Much clearer!
```

Top-Down, Stepwise Refinement

How to solve complex problems:

1. Start with a general description of the problem
2. Break it into smaller sub-problems
3. Keep breaking down until each piece is simple enough to code
4. Solve each piece
5. Put the pieces together

Like building with LEGO blocks:

- Start with big chunks
- Break into smaller pieces
- Build each piece separately
- Connect them together

Example: Student Exam Analysis

Problem: Analyze results of 10 students (each got 1=pass or 2=fail)

Top Level:

"Analyze exam results and decide if instructor gets bonus"

First Refinement:

1. Initialize variables
2. Input the ten results and count passes/failures
3. Print summary and decide on bonus

This is still too general - need more detail!

Second Refinement - More Detail

Initialize variables:

- Set passes to 0
- Set failures to 0
- Set student counter to 1

Input and count:

- While student counter ≤ 10
- Input next result
- If result is 1, add 1 to passes, otherwise add 1 to failures
- Add 1 to student counter

Print and decide:

- Print number of passes and failures
- If more than 8 passed, print "Bonus to instructor!"

Final C Program

```
1  #include <stdio.h>
2
3  int main(void) {
4      // Initialize variables
5      int passes = 0;
6      int failures = 0;
7      int student = 1;
8      // Process 10 students
9      while (student <= 10) {
10         printf("Enter result (1=pass, 2=fail): ");
11         int result;
12         scanf("%d", &result);
13         if (result == 1) {
14             passes = passes + 1;
15         } else {
16             failures = failures + 1;
17         }
18         student = student + 1;
19     } // end while
20     // Display results
21     printf("Passed %d\n", passes);
22     printf("Failed %d\n", failures);
23     if (passes > 8) {
24         printf("Bonus to instructor!\n");
25     }
26     return 0;
27 }
```

Practice Problem 1: Simple Calculator

Problem: Write a program that:

1. Asks user for two numbers
2. Asks for an operation (+, -, *, /)
3. Performs the operation and displays result
4. Handles division by zero

Think about:

- What variables do you need?
- How will you get user input?
- How will you decide which operation to perform?
- What could go wrong?

Try to write the pseudocode first!

Solution: Simple Calculator (Pseudocode)

Pseudocode

1. Input first number
2. Input second number
3. Input operation character
4. If operation is '+'
 - Calculate first + second
 - Display result
5. Else if operation is '-'
 - Calculate first - second
 - Display result
6. Else if operation is '/'
 - If second number is not 0
 - Calculate first / second
 - Display result
 - Else display "Cannot divide by zero"

Introduction to Nested Loops

- A loop inside another loop is called a nested loop.
- Useful for generating patterns and solving matrix-based problems.

Syntax

```
1 for (int i = 0; i < rows; i++) {  
2     for (int j = 0; j < columns; j++) {  
3         // code  
4     }  
5 }  
6
```

Listing 1: Nested Loops Syntax

Example 1: Print a Rectangle of Stars

Code Example

```
1 int rows = 4, columns = 5;
2 for (int i = 0; i < rows; i++) {
3     for (int j = 0; j < columns; j++) {
4         printf("*");
5     }
6     printf("\n");
7 }
8
```

Listing 2: Rectangle of Stars

Example 2: Right-Aligned Triangle of Stars

Code Example

```
1  int rows = 5;
2  for (int i = 1; i <= rows; i++) {
3      for (int j = 1; j <= rows - i; j++) {
4          printf(" ");
5      }
6      for (int k = 1; k <= i; k++) {
7          printf("*");
8      }
9      printf("\n");
10 }
11
```

Listing 3: Right-Aligned Triangle

Example 3: Pyramid of Stars

Code Example

```
1 int rows = 5;
2 for (int i = 1; i <= rows; i++) {
3     for (int j = 1; j <= rows - i; j++) {
4         printf(" ");
5     }
6     for (int k = 1; k <= 2 * i - 1; k++) {
7         printf("*");
8     }
9     printf("\n");
10 }
11
```

Listing 4: Pyramid of Stars

Example 4: Inverted Pyramid of Stars

Code Example

```
1 int rows = 5;
2 for (int i = rows; i >= 1; i--) {
3     for (int j = 1; j <= rows - i; j++) {
4         printf(" ");
5     }
6     for (int k = 1; k <= 2 * i - 1; k++) {
7         printf("*");
8     }
9     printf("\n");
10 }
11
```

Listing 5: Inverted Pyramid of Stars

Example 5: Diamond Shape with Stars

Code Example

```
1 int rows = 5;
2 for (int i = 1; i <= rows; i++) {
3     for (int j = 1; j <= rows - i; j++) {
4         printf(" ");
5     }
6     for (int k = 1; k <= 2 * i - 1; k++) {
7         printf("*");
8     }
9     printf("\n");
10 }
11 for (int i = rows - 1; i >= 1; i--) {
12     for (int j = 1; j <= rows - i; j++) {
13         printf(" ");
14     }
15     for (int k = 1; k <= 2 * i - 1; k++) {
16         printf("*");
17     }
```

Example 6: Hollow Rectangle

Code Example

```
1 int rows = 4, columns = 5;
2 for (int i = 0; i < rows; i++) {
3     for (int j = 0; j < columns; j++) {
4         if (i == 0 || i == rows - 1 || j == 0 || j == columns - 1) {
5             printf("*");
6         } else {
7             printf(" ");
8         }
9     }
10    printf("\n");
11 }
12
```

Listing 7: Inverted Pyramid of Stars

Example 7: Right-Angled Triangle (Hollow)

Code Example

```
1 int rows = 5;
2 for (int i = 1; i <= rows; i++) {
3     for (int j = 1; j <= i; j++) {
4         if (j == 1 || j == i || i == rows) {
5             printf("*");
6         } else {
7             printf(" ");
8         }
9     }
10    printf("\n");
11 }
12
```

Listing 8: Inverted Pyramid of Stars

Example 8: Half Diamond

Code Example

```
1 int rows = 5;
2 for (int i = 1; i <= rows; i++) {
3     for (int j = 1; j <= i; j++) {
4         printf("*");
5     }
6     printf("\n");
7 }
8 for (int i = rows - 1; i >= 1; i--) {
9     for (int j = 1; j <= i; j++) {
10        printf("*");
11    }
12    printf("\n");
13 }
14
```

Listing 9: Inverted Pyramid of Stars

Example 9: Checkerboard Pattern

Code Example

```
1 int size = 8;
2 for (int i = 0; i < size; i++) {
3     for (int j = 0; j < size; j++) {
4         if ((i + j) % 2 == 0) {
5             printf("*");
6         } else {
7             printf(" ");
8         }
9     }
10    printf("\n");
11 }
12
```

Listing 10: Inverted Pyramid of Stars

Example 10: Number Pyramid

Code Example

```
1 int rows = 5;
2 for (int i = 1; i <= rows; i++) {
3     for (int j = 1; j <= rows - i; j++) {
4         printf(" ");
5     }
6     for (int k = 1; k <= i; k++) {
7         printf("%d ", k);
8     }
9     printf("\n");
10 }
11
```

Listing 11: Inverted Pyramid of Stars

Questions & Discussion

Remember

Programming is a skill developed through practice. Every error you make and fix makes you a better programmer!

Remember: Making mistakes is part of learning.
Every expert programmer started where you are now.